



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module A: Tools & Foundations

SQL Foundation Recap

TI / BCS

2025-2026

1

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 2

2

PURPOSE

- Refresh essential SQL knowledge before more advanced topics.
- Align everyone's understanding before moving into complex SQL queries and optimization.
- Build confidence in writing and interpreting basic SQL statements.

Slide 3

3

OBJECTIVES

- Review SQL syntax and structure:** Understand SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, OFFSET, FETCH.
- Introduce aggregation and grouping:** Use GROUP BY, COUNT, SUM, AVG, and filtering with HAVING.
- Understand data retrieval logic:** Master the execution order of SQL clauses.
- Explore joins and table relationships:** Recap how to combine data from multiple tables.
- Practice basic data manipulation:** Use INSERT, UPDATE, DELETE correctly.
- Reinforce knowledge of data types and constraints:** Grasp how columns are defined, and how constraints like PRIMARY KEY, NOT NULL, etc., function.
- Touch on best practices and common mistakes:** Recognize the importance of WHERE in UPDATE/DELETE, and pitfalls like NULL handling.

Slide 4

4

WHAT IS A RELATION DATABASE MANAGEMENT SYSTEM (RDBMS)?

- A **relational database** is a digital database based on the relational model of data and consists of:
 - Tables
 - Columns or Fields
 - Rows or Records
 - Relations
 - Primary Keys
 - Foreign Keys
 - Indexes (technical lookup tables to speed up data retrieval)
- A software system used to maintain relational databases is a **relational database management system** (RDBMS)

Slide 5

5

DATA ORGANISATION HIERARCHY IN POSTGRESQL

Level	Contains	Purpose	Example
Cluster	Databases	PostgreSQL DB server with multiple DBs	databanken.ucll.be:52526
Database	Schemas or tables	A logical collection of schemas, objects, or data for an application	basis_rdbms
Schema	Tables, views, functions,...	A namespace to organize database objects	public, space_travel, tennis
Table	Rows and columns (data)	The actual structured data	celestial_object, visit, player, team

The **search path** is a setting in some SQL databases (like **PostgreSQL**) that determines **which schemas** the database engine will look in **when you run a query that doesn't specify a schema explicitly**. The default search path is public. The search path can be set for a session or user.
Example: **SET search_path TO space_travel, tennis;** (the order is important!)
Be explicit: When possible, write schema.table to avoid ambiguity.
Use search_path carefully in multi-schema environments to avoid querying the wrong table by accident.

Slide 6

6

STRUCTURED QUERY LANGUAGE (SQL)

- SQL is the standard programming language to interact with relational databases.
- Pronounced as “sequel”.
- The language consists of 4 main areas:
 - DDL : DATA DEFINITION LANGUAGE
 - To create and maintain database schemas
 - DML : DATA MANIPULATION LANGUAGE (*)
 - To create, read, update and delete data
 - DCL : DATA CONTROL LANGUAGE
 - To manage authorisations
 - TCL : TRANSACTION CONTROL LANGUAGE
 - To handle transaction processing in applications
- (*) sometimes DML is further decomposed into:
 - DQL : Data Query Language -> Read
 - DML : Data Manipulation Language -> Create, Update, Delete

Slide 7

7

SQL DIALECTS

- Each specific SQL implementation by a software vendor is called a dialect
- Software vendors conform to a set of core SQL features but may implement vendor specific extensions -> limited portability between DBMS solutions
- ISO Standard
 - ANSI-SQL:2023 (previous version 2016)
- Some dialects
 - PostgreSQL
 - MS SQL SERVER : Transact-SQL
 - ORACLE : PL/SQL
 - MySQL/MariaSql
 - SQLite
 - MS Access

Slide 8

8

SELECT STATEMENT

- SELECT** <ExpressionList> : Specifies the **columns** to appear in the resultset
- FROM** <Table / View> : Specifies the **table(s)** & **relations** to be used
- WHERE** <Criteria> : Filters the **rows**
- GROUP BY** <ExpressionList> : Group rows with the **same column values**
- HAVING** <Criteria> : Filters **groups** defined by GROUP BY
- ORDER BY** <Column(s)> : Defines the **order** of the result rows
- OFFSET** <n> : skips the first n rows
- FETCH FIRST** <m> **ROWS ONLY** : Returns the next m rows (after the offset)

The order of clauses in the SQL query cannot be changed

Slide 9

9

SELECT STATEMENT – EXECUTION FLOW

Step	Clause	Purpose
1	FROM	Identify source tables and join them if needed.
2	WHERE	Filter rows before grouping — removes rows that don't meet the condition.
3	GROUP BY	Group remaining rows into sets based on column(s).
4	HAVING	Filter groups — keeps only groups that meet the condition.
5	SELECT	Choose which columns or expressions to output.
6	ORDER BY	Sort the result set.
7	OFFSET	Skip a specified number of rows from the sorted result.
8	FETCH FIRST	Limit the number of rows returned after offset.

Slide 10

10

FROM HUMAN QUESTION TO SQL QUERY

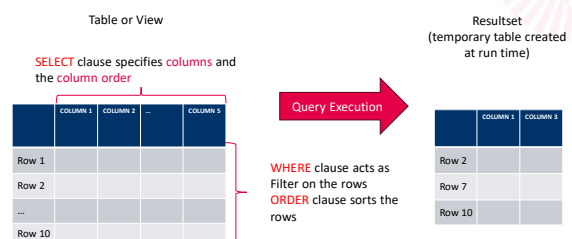
- I would like to know which celestial objects have a diameter of at least 100.000 km.
 - Initial Question: Which celestial objects have a diameter of at least 100.000 km?
 - Refined Question: Provide a list of celestial objects with a diameter of at least 100.000 km, including their objectname, diameter and distance. Sort the list alphabetically by objectname and diameter in ascending order.
- Question type?
- Table(s)?
- Columns?
- Filter?
- Grouping?
- Order?

```
SELECT objectname, diameter, distance
FROM celestial_object
WHERE diameter >= 100000
ORDER BY objectname, diameter;
```

Slide 11

11

RESULTSET = SQL QUERY OUTPUT



Although SELECT is the most common way to produce resultsets, other queries—such as INSERT ... RETURNING, UPDATE ... RETURNING, or CALL for stored procedures—can also return resultsets.

Slide 12

12

SELECT *

- Question: Provide all details of all Celestial objects
- ```
SELECT *
FROM celestial_objects;
```
- The asterisk (\*) returns all columns from the table, in the order they are defined in the table's schema.
  - Attention: the \* should be used only for testing purpose. Columns may be added and/or column names may be changed in a productive system!

13

## SPECIAL RESULTSETS

- **Singleton**: exactly 1 row and 1 or more columns

```
SELECT objectname, distance
FROM celestial_objects
WHERE objectname = 'Mars';
```

| objectname | distance |
|------------|----------|
| Mars       | 227.900  |

- **Single cell**: exactly 1 row and 1 column

```
SELECT objectname
FROM celestial_objects
WHERE satellite_of = 'Earth';
```

| objectname |
|------------|
| Moon       |

- **Empty resultset**: 0 rows and 1 or more columns

```
SELECT objectname, distance
FROM celestial_objects
WHERE distance < 0;
```

| objectname | distance |
|------------|----------|
| (no rows)  |          |

An empty resultset is not the same as a NULL return value

14

## NULL VALUES

- Missing (unknown) data is represented by **NULL** values
- A NULL value is not the same as an empty string ('')
- Any comparison involving a NULL value is unknown
  - Attention with Average calculations -> NULL values are ignored.

15

## FUNCTIONS

- EXTRACT( )
- CAST( )
- TO\_CHAR( )
- ROUND( )
- ...
- Be aware of product-specific features!
- <http://www.postgresql.org/docs/current/interactive/functions.html>

16

## ALIAS

```
SELECT last_name AS name
```

```
SELECT *
FROM player AS s
```

```
SELECT *
FROM player s
```

17

## WHERE CLAUSE

- In the WHERE clause, five basic search conditions can be used
  - **Comparison**: Compares two expressions using operators such as =, <, >, OR, AND, etc
  - **Range**: Checks whether the value is within a specified range of values (BETWEEN)
  - **Set membership**: Checks whether the value matches any value of a given set (IN)
  - **Pattern matching**: Checks whether the expression matches a specified string pattern (LIKE)
  - **NULL check**: Checks whether the expression is a NULL value (IS NULL)

18

## AGGREGATE FUNCTIONS

- Perform a **calculation** on a set of values and **return a single summary value**.
- Aggregate functions: COUNT(), SUM(), AVG(),...
- Question: How many celestial objects are stored in the database?  
`SELECT COUNT(*)  
FROM celestial_object;`
- Question: How many celestial objects are satellites of another object?  
`SELECT COUNT(satellite_of)  
FROM celestial_object;`
- Remark: Aggregate functions ignore NULL values (except COUNT(\*)).

Slide 19

19

## GROUP BY CLAUSE

- Used to **group rows** that have the **same values** into **summary rows**.
- To **perform aggregate calculations** (like COUNT(), SUM(), etc.) **per group** — not just across the whole table.
- **One result row** is created for **each unique combination of values in the GROUP BY columns**.
- **Every column in SELECT must be either grouped or used in an aggregate function.**

Slide 20

20

## GROUP BY

- Question: How many visits were made per celestial object and per duration of stay, ordered by object name and duration of stay?

```
SELECT objectname, duration_of_stay,
 COUNT(*) AS number_of_visits
FROM visit
GROUP BY objectname, duration_of_stay
ORDER BY objectname, duration_of_stay;
```

Slide 21

21

## HAVING CLAUSE

- Question: Which celestial objects, by duration of stay, were visited at least twice?

```
SELECT objectname, duration_of_stay,
 COUNT(journey_nr) AS number_of_visits
FROM visit
GROUP BY objectname, duration_of_stay
HAVING COUNT(journey_nr) >= 2;
```

Slide 22

22

## HAVING CLAUSE

- Question: Which celestial objects, with a stay duration of more than one day, were visited at least twice?

```
SELECT objectname, duration_of_stay,
 COUNT(journey_nr) AS number_of_visits
FROM visit
WHERE duration_of_stay > 1
GROUP BY objectname
HAVING COUNT(DISTINCT journey_nr) >= 2;
```

Slide 23

23

## DISTINCT CLAUSE

- Question: How many unique visits were made per celestial object and per duration of stay?

```
SELECT objectname, duration_of_stay,
 COUNT(DISTINCT journey_nr) AS unique_visits
FROM visit
GROUP BY objectname, COUNT(DISTINCT journey_nr);
```

Slide 24

24

## CASE CLAUSE

```
SELECT objectname,
CASE
 WHEN satellite_of IS NULL THEN 'Sun'
 WHEN satellite_of = 'Sun' THEN 'Planet'
 ELSE 'Satellite'
END AS objecttype
FROM celestial_object;
```

Slide 25

25

## COMBINING TABLES - IMPLICIT

- Implicit join = matches rows between 2 or more tables separated by comma
- Match condition = in WHERE clause
- Question: Which journeys has each client participated in, along with their client number and last name?  

```
SELECT c.client_nr, c.lastname, p.journey_nr
FROM client c, participation p
WHERE c.client_nr = p.client_nr
```

The WHERE clause is important as without a cartesian product will be generated. A cartesian product will show each row in the left table once for every row in the right table (CROSS JOIN). This may have an important impact on performance.

Slide 26

26

## COMBINING TABLES - EXPLICIT

- The clause **JOIN** explicitly matches rows between 2 or more tables
- Different types: INNER, LEFT, RIGHT, FULL OUTER, CROSS JOIN
- Match condition: in ON clause
- Question: Which journeys has each client participated in, along with their client number and last name?  

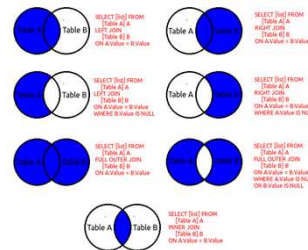
```
SELECT c.client_nr, c.lastname, p.journey_nr
FROM client c INNER JOIN participation p ON c.client_nr = p.client_nr;
```

The ON operator is important as without a cartesian product will be generated. A cartesian product will show each row in the left table once for every row in the right table, This may have an important impact on performance.

Slide 27

27

## JOIN TYPES



Slide 28

28

## JOIN TYPES

- Question: List all clients with their client number, last name, and the journeys they participated in — including clients who have not participated in any journey.

```
SELECT c.client_nr, c.lastname, p.journey_nr
FROM client c LEFT OUTER JOIN participation p
ON c.client_nr = p.client_nr;
```

Slide 29

29

## JOIN TYPES

- Question: Show all celestial objects along with their diameter, distance, and any journeys that visited them — including those that have never been visited.

```
SELECT co.objectname, co.diameter, co.distance, v.journey_nr
FROM visit v
RIGHT OUTER JOIN celestial_object co ON v.objectname = co.objectname;
```

Slide 30

30

## JOIN TYPES

- Question: Which celestial objects in the database have never been visited?

```
SELECT co.objectname
FROM celestial_object co
LEFT OUTER JOIN visit v ON co.objectname = v.objectname
WHERE v.objectname IS NULL;
```

Slide 31

31

## COMBINING RESULT SETS - UNION CLAUSE

- The **UNION** clause combines (appends) the results of two or more **SELECT** queries into a single result set, eliminating duplicate rows by default.
- The **UNION ALL** operator retains duplicate rows in the final result set.
- Rules for the queries when using the **UNION** operator:
  - The queries have the same number of columns.
  - The corresponding columns must have compatible data types.
  - The column names of the first query determine the column names of the combined result set.
  - If you use the **GROUP BY** and **HAVING** clauses, they will be applied to each query, not the final result set.
  - If you use the **ORDER BY** clause, it will be applied to the combined result set, not the individual result set.

Slide 32

32

## COMBINING RESULT SETS - UNION CLAUSE

- Question: Combine a list of all stars (objects that are not satellites) with a list of planets and large moons (satellites with diameter > 3000 km), without duplicates. Add a column to reference the source query.
- ```
SELECT objectname, diameter, distance, 'Star' AS source
FROM celestial_object
WHERE satellite_of IS NULL
UNION
SELECT objectname, diameter, distance, 'Planet or large moon'
FROM celestial_object
WHERE satellite_of IS NOT NULL AND diameter > 3000 ;
```

Slide 33

33

CREATE TABLE

```
CREATE TABLE schema_name.table_name (
    column_name1 data_type [constraints],
    column_name2 data_type [constraints],
    ...
);
```

- Always define a primary key!
- Use **SERIAL** or **GENERATED** for auto-incrementing IDs
- Use meaningful column names and types

Slide 34

34

CREATE TABLE

Constraint	Description
PRIMARY KEY	Uniquely identifies each row in a table. Automatically implies NOT NULL and UNIQUE .
NOT NULL	Ensures the column must have a value — it cannot be left empty.
DEFAULT	Sets a default value for the column if no value is provided during insertion.
UNIQUE	Ensures all values in the column (or group of columns) are distinct .
CHECK	Validates that values meet a specific condition or expression .
FOREIGN KEY	Ensures the value in a column matches a value in another table's primary key — maintains referential integrity .

Slide 35

35

CREATE TABLE

```
CREATE TABLE space_travel.visit (
    journey_nr      numeric(4) NOT NULL,
    objectname      varchar(10) NULL,
    sequence_number numeric(2) NOT NULL,
    duration_of_stay numeric(4) NULL,
    CONSTRAINT visit_pkey PRIMARY KEY (journey_nr,
    sequence_number),
    CONSTRAINT journey_fk FOREIGN KEY (journey_nr) REFERENCES
    space_travel.journey(journey_nr),
    CONSTRAINT object_fk FOREIGN KEY (objectname) REFERENCES
    space_travel.celestial_object(objectname)
);
```

Slide 36

36

GRANT/REVOKE

- Control **user permissions** on database objects like database, schemas, tables, roles etc
- GRANT - Give permissions
GRANT privilege [, ...]
ON object
TO user [, ...];
- REVOKE – Remove permissions
REVOKE privilege [, ...]
ON object
FROM user [, ...];

Slide 37

37

GRANT/REVOKE

Privilege	Applies To
SELECT	Tables, views
INSERT	Tables
UPDATE	Tables
DELETE	Tables
USAGE	Schemas, sequences
EXECUTE	Functions
ALL	All applicable rights

Use GRANT ALL PRIVILEGES to grant everything applicable.
Use with care: improper permissions can expose sensitive data.

Slide 38

38

GRANT / REVOKE

- GRANT SELECT
ON TABLE space_travel.visit
TO student;
- REVOKE UPDATE, INSERT
ON TABLE space_travel.celestial_object
FROM rxxxxxx;

Slide 39

39

INSERT STATEMENT

- The INSERT statement is used to add new rows in a table.
INSERT INTO tableName
(column_1, column_2,...,column_n)
VALUES (value_1, value_2,...,value_n)
- It is allowed to specify multiple values for one column. For each new value a new row will be created.
INSERT INTO tableName (Column_1)
VALUES (Value_1a), (Value_1b),..., (Value_1n)
- The **DEFAULT VALUES** clause allows to create a new row with the default values specified in the CREATE TABLE statement

Slide 40

40

INSERT STATEMENT

- The **INSERT INTO SELECT** statement allows to **copy data from one table to another table**. The corresponding table columns of the two tables should have compatible datatypes.
- Copy all columns from one table into another table
INSERT INTO tableName2
SELECT *
FROM tableName1
[WHERE condition];
- Copy some columns from one table into another table
INSERT INTO tableName2 (column_1, column_2,..., column_n)
SELECT column_1, column_2,..., column_n
FROM tableName1
[WHERE condition];

Slide 41

41

INSERT STATEMENT

- Question: Add a new client Steve Jobs, including all relevant personal details.

```
INSERT INTO client (client_nr, lastname, firstname, birthdate)  
VALUES (127, 'Jobs','Steve','1955-02-24');
```

Slide 42

42

INSERT STATEMENT

- Question: Suppose we want to create a reporting table called `visited_large_objects` that stores all celestial objects visited at least once and with a diameter over 5000 km

```
CREATE TABLE space_travel.visited_large_objects (  
  objectname TEXT PRIMARY KEY,  
  diameter NUMERIC,  
  distance NUMERIC);  
  
INSERT INTO visited_large_objects (objectname, diameter, distance)  
SELECT DISTINCT co.objectname, co.diameter, co.distance  
FROM celestial_object co  
INNER JOIN visit v ON co.objectname = v.objectname  
WHERE co.diameter > 5000;
```

Slide 43

43

UPDATE STATEMENT

- The **UPDATE** statement is used to modify existing rows in a table.

```
UPDATE tableName  
SET column_1 = value_1, column_2 = value_2, ... , column_n = value_n  
[WHERE condition]
```

- Important: the WHERE clause determines which rows will be updated. If no WHERE clause is specified, ALL rows are updated!

o Use WHERE 1=0 as a temporary safety mechanism during development, especially in UPDATE/DELETE statements — but always remember to replace it when your query is ready to run.

Slide 44

44

UPDATE STATEMENT

- The **UPDATE FROM** statement allows to update data in one table based on data from another table.

```
UPDATE targetTableName AS t  
SET column_1 = s.value_1,..., column_n = s.value_n  
FROM sourceTableName AS s  
[WHERE condition];
```
- The sourceTableName can be a SELECT query
- In PostgreSQL, when you alias the target table in an UPDATE, you **can** use the alias in the WHERE and FROM clauses **but** in the SET clause, PostgreSQL prefers unqualified column names for the target table's columns.

Slide 45

45

UPDATE STATEMENT

- Question: Increase the duration of all visits to Mars by 1 day.

```
UPDATE visit  
SET duration_of_stay = duration_of_stay + 1  
WHERE objectname = 'Mars';
```

Slide 46

46

UPDATE STATEMENT

- Question: After recalculating planetary data, astronomers discovered that all visited celestial objects beyond 1.000.000 km should have their recorded distance reduced by 2% to reflect more accurate measurements.

```
UPDATE celestial_object AS co  
SET distance = co.distance * 0.98  
FROM visit v  
WHERE co.objectname = v.objectname  
AND co.distance > 1000000;
```

Slide 47

47

DELETE STATEMENT

- The **DELETE** statement is used to delete existing rows from a table.

```
DELETE FROM tableName  
[WHERE condition];
```

- Important: the WHERE clause determines which rows will be deleted. If no WHERE clause is specified, ALL rows are deleted!

o Use WHERE 1=0 as a temporary safety mechanism during development, especially in UPDATE/DELETE statements — but always remember to replace it when your query is ready to run.

Slide 48

48

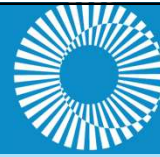
DELETE STATEMENT

- QUESTION: Which command will delete the client entry corresponding to Steve Jobs from the client table?

```
DELETE FROM client
WHERE lastname = 'Jobs' AND firstname = 'Steve'
;
```

Slide 49

49



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module A: Tools & Foundations Data Modelling Recap

T1 / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

50

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 51

51

PURPOSE

- Understand the principles and importance of data modelling in information systems.
- Develop the ability to represent real-world data requirements effectively.
- Bridge the gap between business requirements and technical database design.

Slide 52

52

OBJECTIVES

- Differentiate between conceptual, logical, and physical data models.
- Apply entity-relationship modelling techniques to real-world scenarios.
- Translate data models into database schemas (e.g., PostgreSQL).

53

Slide 53

53

DATA MODELLING IN CONTEXT

- What is a data model?
- Why is data modelling important?
- What makes a good data model?
- Where does data modelling fit in application development?
- What are the design stages and deliverables?

54

Slide 54

54

WHAT IS MODELLING?



Slide 55

55

MODELLING BASED ON USER REQUIREMENTS



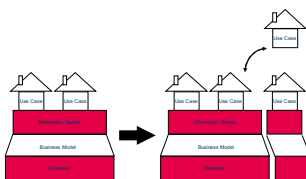
E.g. The system must be able to automatically create a purchase requisition when a minimum stocklevel is reached for a raw material.
The system must be able to automatically convert 1 purchase requisition into 1 purchase order for purchase requisitions that were automatically created.



Slide 56

56

MODELLING BASED ON BUSINESS CONTEXT



E.g. Materials requirements planning is applied for all raw materials, and semi-finished products that are used within the manufacturing of our commercial products. The material requirements planning process is governed by the Supply Chain organisation. The key business objectives are to reduce working capital while respecting customer service levels.



Slide 57

57

WHAT IS DATA MODEL?

- A data model describes what data will be held in a database and how it will be organised.
- Data models can be represented in many different ways.
- In a relational database data is organised into tables.



Slide 58

58

MODELLING IS A DESIGN ACTIVITY

- We have the choice of what symbols or codes we use to represent real-world facts
- Usually there is more than one way to organise data into tables and columns
- Usually requirements are incomplete or loose and might result in different solutions
- We have some choice in what part of the system will handle each business requirement



Slide 59

59

WHY IS DATA MODEL(ING) IMPORTANT?

- Small changes to the data model might have an impact on the system as a whole
 - A well-designed data model can make programming simpler and cheaper
 - Poor data organisation can be (very) expensive
- A Data model is a strong tool for expressing requirements and capabilities
- A Data model plays a key role in achieving good data quality



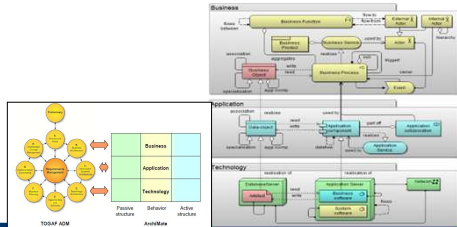
Slide 60

60

Data modelling is a small part of the total system specification but it has a high impact on the quality and useful life of the system

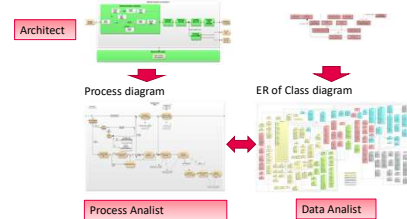
DATA MODELLING IN THE CONTEXT OF: ENTERPRISE ARCHITECTURE

- Archimate as language to model Architecture artefacts



61

DATA MODELLING IN THE CONTEXT OF: APPLICATION DEVELOPMENT



62

DATA MODELLING IN THE CONTEXT OF: PACKAGED SOLUTIONS (ERP, CRM,...)

- Identify data requirements for fit for purpose evaluation and configuration
- Legacy data migration
- Data Governance & Data lineage
- Master Data Alignment / Integration

63

DATA MODELLING IN THE CONTEXT OF: DATA GOVERNANCE

- Data is a corporate asset and needs to be managed accordingly
- Data Governance provides the guidelines, definitions and accountability about data management
- A data model can help define:
 - Who is the data owner (accountability)?
 - Who is using data?
 - What is the lineage and traceability?
 - How can I audit data?

64

DATA MODELLING IN THE CONTEXT OF: METADATA MANAGEMENT

- Metadata (metacontent) is defined as the data providing information about one or more aspects of the data, such as:
 - Means of creation of the data
 - Purpose of the data
 - Time and date of creation
 - Creator or author of the data
 - Location on a computer network where the data was created
 - Standards used

65

DATA MODELLING IN THE CONTEXT OF: BUSINESS INTELLIGENCE

- Data Warehouse = specialised database
 - Organized around major subjects, such as customer, product, sales.
 - Focusing on the modelling and analysis of data for decision makers, not on daily operations or transaction processing.
 - Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process.
 - Constructed by integrating multiple, heterogeneous data sources.
 - Data cleansing and data integration techniques are applied.

66

DATA MODELLING IN THE CONTEXT OF: MASTER DATA MANAGEMENT

- Master data is the high value information used across departments, applications and business processes.
 - Example: customers, products, vendors, assets, etc.
- Master Data Management is the practice of providing an enterprise-wide "single view" also known as "golden record"
- Data modelling will support to:
 - Improve the consistency and trustworthiness of data across the enterprise.
 - Improve the operational efficiency by enabling cross-functional collaboration.
 - Remove the duplicated data management activities across departments.
 - Provide a solid foundation for compliance and risk management.



Slide 67

67

WHAT MAKES A GOOD DATA MODEL?

- Completeness
- Klik om de modelstijlen te bewerken
- Tweede niveau
- Nonredundancy
 - Vierde niveau
 - Vijfde niveau
- Enforcement of business rules
- Data reusability

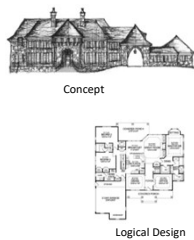


Slide 68

Design a model providing the best balance between possibly conflicting objectives

68

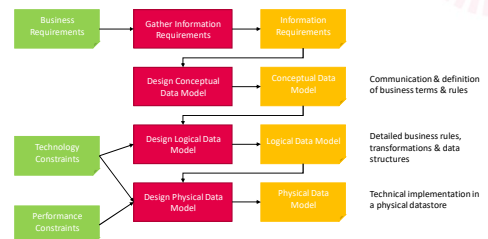
DIFFERENT LEVELS OF ABSTRACTION



Slide 69

69

DESIGN STAGES AND DELIVERABLES



Slide 70

70

THE CONCEPTUAL DATA MODEL (CDM)

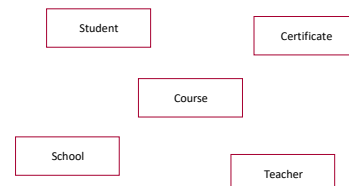
- What**
Representation of real-world facts using simple graphical images
- Audience**
Business people
- Purpose**
Facilitate communication
- Abstraction level:**
High enough to be intuitive
Detailed enough to capture definitions and rules to create database systems



Slide 71

71

SOME EXAMPLES OF CDM



Slide 72

Boxes representing real-world facts

72

SOME EXAMPLES OF CDM

Name	Definition
Customer	The recipient and buyer of our product
Order	A contract to buy a quantity of product at a specified price
Product	Anything we buy, sell, stock, move or make

Business Terms & Definitions describing real-world facts

73

HOW TO BUILD MODELS?

- Top Down
- Bottom Up
- Middle Out

74

CDM: TERMINOLOGY

- Business oriented – “business friendly”
- 3 basic concepts
 - Entity Classes
 - Relationships
 - Attributes
- Process is called Entity Relationship modelling or E-R modelling (Peter Chen)
- The outcome is an E-R model or Conceptual model
- The diagram is an E-R Diagram or ERD

75

ENTITY CLASSES OR ENTITY TYPE

- Classification of the types of objects found in the real world such as persons, places, things, concepts and events of interest to the business (DAMA: Dictionary of Data Management)
- Represented by boxes on a diagram
- Generally implemented as tables
- Entity Class is collection of Entities
 - Entity Class: Student
 - Entity: Joe Bar
- Name of the Entity Class must be singular

76

ENTITY CLASSES ARE THE "NOUNS" OF THE ORGANISATION

- Who: Customer, Student, Employee
- What: Material, Course, Product, Service
- Where: Location, Address
- When: Fiscal Year, Period
- Why: Order, Credit, Claim, Transaction
- How: Agreement, Invoice, Certificate

77

NAMING CONVENTIONS

Use	Avoid Using
Account	Accounts
Product	Product Catalog
Historical Transaction	Transaction History
Scheduled Visit	Visiting Schedule
Material Item	Component

78

ENTITY CLASS DEFINITIONS

- Entity Class names must be supported by definitions!!!
- A good definition will answer two questions:
 - What distinguishes instances of this entity class from instances of another entity class?
 - What distinguishes one instance from another?



Slide 79

79

SAMPLES

- A Prospect is a person or organisation who does not own any of our products and who is potentially interested in purchasing one or more of our products.
- A Customer is a person or organisation who has purchased at least one of our products and has an active account.
- A Sales Order is a contract between our Sales Organization and a Customer for supply of specified goods and/services over a specified timeframe and in an agreed upon quantity or unit.



Slide 80

80

EXERCISE

- Maintenance workers inspect equipments in the Factory on a periodical basis. Any defect or strange behaviour is registered on a notification document and handed over to the Maintenance manager. After careful consideration the Maintenance manager might issue a maintenance order and authorises spare parts to be collected from the spare part warehouse.
- Identify Entity Classes



Slide 81

81

RELATIONSHIPS ARE THE "VERBS"

- Relationships define data-centric business rules
- Notation: line between Entity Classes



- Definition: clockwise and defined in both directions (to avoid separate meanings)



Slide 82

82

CARDINALITY: HOW MANY

- Each Customer can make one or more Purchases
- A Purchase can be made by one and only one Customer

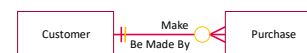


Slide 83

83

OPTIONALITY: MUST OR MAY

- Each Customer **may** make one or more Purchases
- A Purchase **must** be made by one and only one Customer

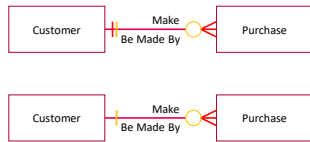


Slide 84

84

SIMPLIFICATION

- Cardinality 1 is often not mentioned in an ER Diagram



Slide 85

85

TYPICAL RELATIONSHIPS

- One-to-One (1:1)



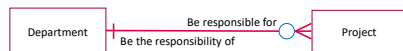
- Each Department must be managed by one Manager
- Each Manager may manage a Department

Slide 86

86

TYPICAL RELATIONSHIPS

- One-to-Many (1:N)



- Each Department may be responsible for one or more Projects
- Each Project must be the responsibility of one Department

Slide 87

87

TYPICAL RELATIONSHIPS

- Many-to-Many (M:N)



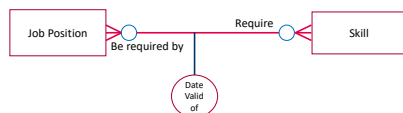
- Each Job Position may require one or more Skills
- Each Skill may be required by one or more Job Positions

Slide 88

88

TYPICAL RELATIONSHIPS

- Many-to-Many



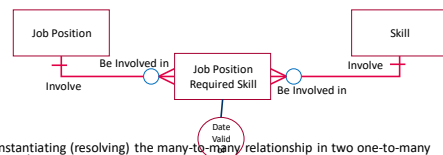
- How to model attributes (eg "date valid of") that are relationship dependent?

Slide 89

89

TYPICAL RELATIONSHIPS

- Many-to-Many



- By instantiating (resolving) the many-to-many relationship in two one-to-many relationships
- The new Entity Class is often called an associative, intersection, relationship or resolution Entity Class

Slide 90

90

TYPICAL RELATIONSHIPS

- Two relationships



- Relationship 1:
 - Each Employee must hold one Position
 - Each Position may be held by one Employee
- Relationship 2:
 - Each Employee may act in one or more Positions
 - Each Position may be acted in by one Employee

Slide 91

91

TYPICAL RELATIONSHIPS

- Self-referencing or recursive one-to-one



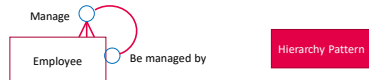
- Each Inspection may precede one Inspection
- Each Inspection may follow one Inspection

Slide 92

92

TYPICAL RELATIONSHIPS

- Self-referencing or recursive one-to-many



- Each Employee may be managed by one Employee
- Each Employee may manage one or more Employees

Slide 93

93

TYPICAL RELATIONSHIPS

- Self-referencing or recursive many-to-many



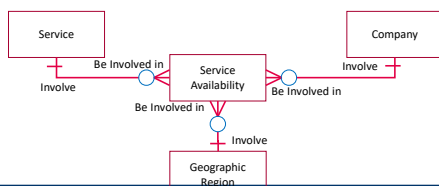
- Each Geographic Region may be made up of one or more Geographic Regions
- Each Geographic Region may be contained in one or more Geographic Regions

Slide 94

94

TYPICAL RELATIONSHIPS

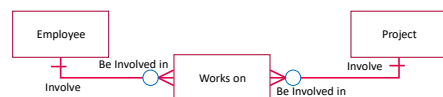
- More than 2 relationships
- Example: Record in the system a Service that is provided by a Company in a specific Geographic Region



Slide 95

95

TYPICAL RELATIONSHIPS - AGGREGATION

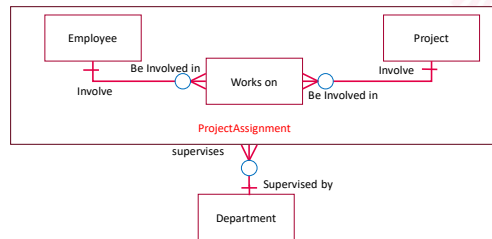


How could we model the requirement that a Department should supervise the assignment of an Employee on a Project?

Slide 96

96

TYPICAL RELATIONSHIPS - AGGREGATION



Instead of directly relating Department to both Employee and Project, we can **aggregate** the "Works on" relationship into a higher-level entity: the assignment of an Employee on a Project, we can call it "ProjectAssignment" and add a relation with Department to it.

Slide 97

97

TYPICAL RELATIONSHIPS - AGGREGATION

- **Aggregation** is a **higher-level abstraction** that allows you to treat a **relationship** as though it were an **entity**.
- It is used when you need to express a relationship **between a relationship and an entity**. In other words, aggregation is a way of "abstracting" or "boxing up" a relationship into a higher-level entity, so that it can participate in another relationship.

38

98

EXERCISE

- Each Inspector may perform one or more Inspections
- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

99

Slide 99

99

EXERCISE

- Each Person may be the beneficiary of one or more Insurance Policies
- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

30

de 100

100

EXERCISE

- Each Insurance Policy must insure one Person
- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

101

Slide 101

101

EXERCISE

- Each Employee may formally report to one Employee
- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

22

de 102

102

EXERCISE

A department employs many persons. A person is employed by, at most, one department.

- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

Slide 103

103

EXERCISE

An author may write many books. A book may be written by many authors.

- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

Slide 104

104

EXERCISE

A team consists of many players. A player plays for only one team.

- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

Slide 105

105

EXERCISE

A lecturer teaches, at most, one course. A course is taught by exactly one lecturer.

- Create a data model for each business rule. If you don't think enough information is available to define either of these, then state an assumption that makes it clear.

Slide 106

106

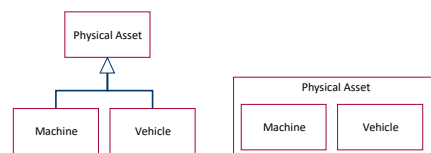
SUBTYPES AND SUPERTYPES

- Consider the following:
 - Our company owns Physical assets
 - All Physical Assets are identified with a unique identifier however for Vehicles we use the car registration number as an alternate identifier.
 - An Employee may be the driver of a Vehicle. But a Machine which is also a Physical Asset can never have a driver.
 - Not all Employees are authorised to drive a Vehicle.
- How would we model these rules?

Slide 107

107

SUBTYPES AND SUPERTYPES 2 REPRESENTATIONS

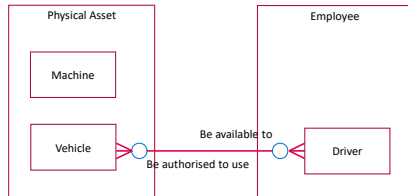


The Entity Types Machine and Vehicle are subtypes of the Entity Type Physical Asset

Slide 108

108

SUBTYPES AND SUPERTYPES



Slide 109

109

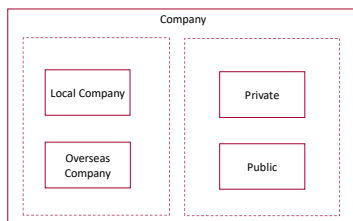
SUBTYPES AND SUPERTYPES

- Mechanism for generalising/specialising Entity Classes
- Useful in the following cases
 - Different identifiers
 - Different attribute(s)
 - Different relationships
 - Different processes (wholesales vs retail orders)
 - Help clarifying the model for the user
- Used in the data modelling process, rather than structures that will be implemented

Slide 110

110

SUBTYPES AND SUPERTYPES MULTIPLE PARTITIONS



Slide 111

111

SUBTYPES AND SUPERTYPES

Definitions	Attributes
• A subtype Entity Class inherits the definition of its supertype Entity Class • Specify what differentiates the subtype from its sibling subtypes	• A subtype Entity Class inherits the attributes of its supertype Entity Class • Specify what attributes differentiates the subtype from its sibling subtypes and supertype

Slide 112

112

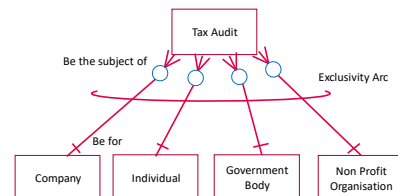
SUBTYPES AND SUPERTYPES

- Important Rule
- Only nonoverlapping, exhaustive Subtypes
 - A company cannot be both a public and private company = nonoverlapping
 - A company must be either a public or a private company, nothing else = exhaustive

Slide 113

113

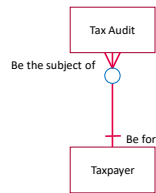
MUTUALLY EXCLUSIVE RELATIONSHIPS



Slide 114

114

MUTUALLY EXCLUSIVE RELATIONSHIPS



Mutually exclusive relationships can be generalised

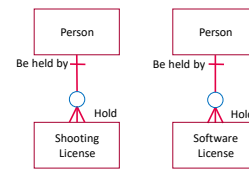


Slide 115

115

TRANSFERABILITY

- Consider the following CDM's



Slide 116

116

TRANSFERABILITY

- Two models have same structure, however
 - Software license may be transferred from one person to another
 - Shooting license cannot be transferred from one person to another
 - Every time someone qualifies for a Shooting license a new one is issued.
- Difference in transferability has important consequences



Slide 117

117

TRANSFERABILITY

- Unique Identifier
 - Shooting License: License No, Person Id
 - Software License: License No, Person Id
 - => In the second case is the Identifier not stable as the Person Id would change if Software license is transferred
- History / Audit trail
 - History needs to be modeled for Software licenses, not for Shooting licenses



Slide 118

118

TRANSFERABILITY

- Notation: no official notation



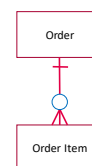
- Definition:
 - Each Shooting License must be owned by one Person who must not change over time.
 - Each Software License must be owned by one Person who may change over time



Slide 119

119

DEPENDENT AND INDEPENDENT ENTITY CLASSES



A **Dependent Entity Class** can only exist in conjunction with instances of another Entity Class and cannot be transferred between instances of that other Entity Class

Order item is a Dependent Entity Class

It cannot exist outside orders and cannot be transferred between orders.



Slide 120

120

ATTRIBUTES THE NUTS AND BOLTS OF DATA

- Define what we want to know about Entity Classes
- Not usually shown on an ER diagram
- Generally implemented as columns in tables
- Terminology:
 - Conceptual Data Model: Term “Attribute” is used
 - Logical & Physical Data Model: Term “Column” is used



Slide 121

121

ATTRIBUTE DEFINITION

- Unambiguously defined at the end of the modelling phase
- Answer the following questions:
 - What does it mean to assign a value to this attribute?
 - What does each value that can be assigned to this attribute mean
- Example: Date of birth
 - Assignment of a value to the Date of Birth for an instance of an Employee is a record of the calendar date on which that Employee was born.



Slide 122

122

ATTRIBUTE DEFINITION

- Conceptual Data Model
 - What type of attribute in business-friendly words
 - Any important business rules
- Logical Data Model
 - What type of column
 - Whether it forms part of the key
 - What business rules it is subject to, whether it is mandatory and the range of allowed values
 - What is the likelihood that constraints will change during the life of the system
 - Formats to be used



Slide 123

123

ATTRIBUTE TYPES - CLASSIFICATION

- Identifier: identify Entity instances
 - Customer Id, Product Number
- Category: hold defined set of values
 - Customer Type, Payment Method, Good Customer Flag
- Quantifier: attribute on which some maths and comparisons can be performed
 - Order Quantity, Unit Price, Order Date
- Text Item: any string of characters
 - Customer Name, Vendor Address



Slide 124

124

ATTRIBUTE TYPES

- Integer
- Date
- Float
- Char (n)
- Varchar (n)
- Timestamp
- Boolean
- ...
- Are DBMS dependent
- In a Conceptual Model the emphasis should be more on how the business intends to use the attribute than how it might be implemented in a particular DBMS



Slide 125

125

IDENTIFIER TYPE

- In a relational model **every row** in a table must be **uniquely defined by an Identifier or Primary Key**.
- During Conceptual modelling it is important to focus on how the business uniquely defines an Entity Instance from another Entity Instance.
- If no uniqueness can be found based on the defined attributes we can invent a key – this can be deferred until the Logical data modelling stage.



Slide 126

126

IDENTIFIER TYPE

- System generated
 - Sequentially generated number to identify each new instance
 - Pre-defined number ranges
 - Nonnumeric (eg airline reservation)
 - Tie-breaker (eg Orderline number)
- Administrator defined
 - Unique codes defined by an Administrator
 - Typical for low volume Entity Classes, mainly master data
- Externally defined
 - ISO defined codes (Currency, Language,...)
 - Zip codes



Slide 127

127

IDENTIFIER TYPE

- Foreign Key (<> Identifier)
 - Principal means of linking data from different tables. A Foreign Key supports the implementation of the relationships defined in a CDM into relations between tables. Foreign Keys are modelled at logical/physical level.
 - Each Foreign Key value has a matching Primary Key value = Referential Integrity.
 - A Foreign Key can only be part of a Primary Key when the



Slide 128

128

CATEGORY TYPE

- Administrator or externally defined
- hold defined set of values
- Sample:
 - "Male", "Female"
 - "Credit Card", "Paypal", "Debet Card"
 - "Yes", "No"



Slide 129

129

QUANTIFIER TYPE

- Forms
 - Count : How many?
 - Dimension: How long, How wide,...? + Unit
 - Currency Amount: How much...? + Unit
 - Factor: Division by one fact by another + Unit
 - Single Time Point: When...? Single event
 - Recurring Time Point: When...? Recurring event
 - Interval: For how long...? + Unit
 - Location: Where...?
- Identify and document what level of accuracy is required by business
- Quantifiers with units
 - Express all instances of the attribute are expressed in same unit
 - Use additional attribute for the unit
 - Define conversion rules



Slide 130

130

CLASSIFICATION BENEFITS

- Better understanding by business stakeholders of what is proposed
- Better understanding by process modellers of how each attribute can be used
- Simplify the definition of attributes
- Standardisation



Slide 131

131

ATTRIBUTE DOMAIN

- A Domain defines the possible values of an attribute
 - <> set of values stored in specific column
 - A same set of values can have different meanings, these meanings are important to identify during data modelling.
- Identifying attribute domains supports the quality of the data model
 - Only attributes in the same attribute domain can be compared
 - The value in an attribute can only be assigned to another attribute belonging to the same attribute domain
 - Each attribute domain allows some operations (comparison, order by, first value,...)



Slide 132

132

ATTRIBUTE DOMAIN – HOW TO?

- Each attribute that is used as identifier of an Entity Class should be assigned its own attribute domain
- Each category attribute should have its own attribute domain assigned unless it share the same possible values and meanings with another one
- All quantifier attributes of the same attribute type can be assigned the same attribute domain
- Text attributes with different meanings should be assigned different attribute definitions

Slide 133

133

EXAMPLE

Attribute Type	Domain	Attribute
Identifier	System-generated identifier	Customer Number
	Administrator defined identifier	Product Code
	Pre-identifier	Order Line Number
Category	Order Line Number	Product Category
	Product Category	Payment Method
	Payment Method	Preferred Payment Method
Quantifier	Count	Order Quantity
	Currency Amount	Purchase Price
	Currency Amount	Sales Price
	Specific Time Point	Purchase Date
Text	Date	Invoice Date
		Requested Delivery Date
	Product Name	Product Name
	Address	Ship to Address
	Address	Registered Business Address

Slide 134

134

BUSINESS RULES

- Systems and Databases contain and enforce rules about the business
- A business rule or constraint may be:
 - Information that something has happened
 - Information how the system should behave in a particular situation

Slide 135

135

BUSINESS RULES TYPES

- Data Rules
 - Data Validation Rules: what data may be recorded in a database and what changes may be made.
(example: An employees employment status should either be permanent or temporary)
 - Cardinality and Referential Integrity are data validation rules
 - Data derivation Rules: specify the calculation methods for deriving data on forms, reports, etc
(example: Annual Gross Salary calculation method)

Slide 136

136

BUSINESS RULES TYPES

- Business Process Rules
 - Rules that define what processing the system has to do in particular circumstances
(Example: if the inventory quantity of material A drops below 20 units then the material should be replenished)
 - Additional attributes might need to be modelled in the CDM to support process rules

Slide 137

137

BUSINESS RULES

- Defined in E-R Diagram
 - Referential Integrity rules
 - Cardinalities of Relationships
 - Mandatory/Optional Relationships
 - Subtyping
 - Reference tables
- Defined in CDM
 - Defined in E-R Diagram +
 - Optionality of attributes
 - Attribute type
 - Attribute Domain

Slide 138

138

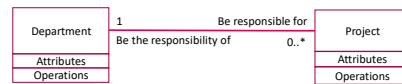
BUSINESS RULES

- Other Rules
 - Attribute value constraints
 - Attribute constraints dependent on values of other attributes of the same Entity instance
 - Attribute constraints dependent on values of other attributes of different Entity instances
 - Restrictions on updatability

Slide 139

139

OTHER NOTATIONS UML – CLASSES AND ASSOCIATIONS

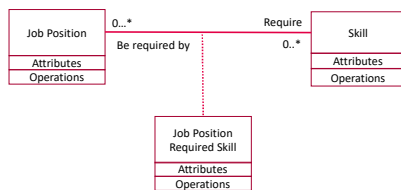


Indicator	Meaning
0..1	Zero or one
1	One only
0..*	Zero or more
*	Zero or more
1..*	One or more
3	Three only
0..5	Zero to Five

Slide 140

140

OTHER NOTATIONS UML – ASSOCIATION CLASS



Slide 141

141

OTHER NOTATIONS UML - AGGREGATION



An association with an aggregation relationship indicates that one class is a part of another class. In an aggregation relationship, the child class instance can outlive its parent class.

Slide 142

142

OTHER NOTATIONS UML - COMPOSITION



The composition aggregation relationship is just another form of the aggregation relationship, but the child class's instance lifecycle is dependent on the parent class's instance lifecycle

When the Company instance is removed, the Department instance is automatically removed as well. Another important feature of composition aggregation is that the part class can only be related to one instance of the parent class.

Slide 143

143

ENTITY CLASS DEFINITION TEMPLATE

Entity Class Name	Superclass Name	Key
Entity Class Definition		
Business Rules / Constraints		
Attribute Name	Attribute Type	Attribute Domain
Attribute Name	Attribute Type	Attribute Domain
Attribute Name	Attribute Type	Attribute Domain
Attribute Name	Attribute Type	Attribute Domain
Attribute Name	Attribute Type	Attribute Domain
Relationship Definition		
Relationship Name	Relationship Type	Relationship Domain
Relationship Name	Relationship Type	Relationship Domain
Relationship Name	Relationship Type	Relationship Domain
Relationship Name	Relationship Type	Relationship Domain
Relationship Name	Relationship Type	Relationship Domain

Slide 144

144

CDM => LDM => PDM

- **Conceptual Data Model (CDM)**
 - High-level, business-oriented view of data.
 - Focuses on entities, relationships, attributes.
 - Independent of technology or DBMS.
- **Logical Data Model (LDM)**
 - Refines the conceptual model into structures understood by databases.
 - Entities => Tables, Attributes => Columns, Relationships => Foreign Keys.
 - Still platform-independent, normalized.
- **Physical Data Model (PDM)**
 - Implements the logical model in a specific DBMS (e.g., PostgreSQL).
 - Defines exact data types, indexes, constraints, storage details.
 - Performance tuning and DBMS-specific optimizations.



Slide 145

145

CONVERTING CDM INTO LDM

- **Entities => Tables**
 - Each **entity type** in the conceptual model becomes a **table** in the logical model.
 - Strong entities map directly to independent tables.
 - Weak entities become tables, but include the primary key of the owner entity as part of their own primary key.
- **Attributes => Columns**
 - Each **attribute** becomes a **column** in the corresponding table.
 - Multi-valued attributes are transformed into separate tables (with foreign key references).
 - Composite attributes are broken down into their component **simple attributes**.



Slide 146

146

CONVERTING CDM INTO LDM

- **Identifiers => Primary Keys**
 - The **unique identifier(s)** of an entity become the **primary key** of the corresponding table.
 - If no natural key exists, a surrogate key is introduced.
- **Relationships => Foreign Keys / Tables**
 - **1:1** : Add a foreign key in one of the tables (the side with optional participation).
 - **1:N** : Add the primary key of the "one" side as a foreign key in the "many" side.
 - **M:N** : Create a new **associative (bridge) table** with foreign keys referencing both entities.



Slide 147

147

CONVERTING CDM INTO LDM

- **Generalization / Specialization => Table Structures**
 - **Options:**
 - **Single Table Inheritance (One Table):** All supertype and subtype attributes in one table, with a type discriminator.
 - **Multiple Table Inheritance (Multiple Tables):** One table for supertype, one for each subtype.
 - **Separate Tables:** Each subtype becomes an independent table (repeating supertype attributes).
- **Constraints => Logical Specifications**
 - **Participation & cardinalities** (min/max) become **NOT NULL constraints** or uniqueness checks.
 - **Business rules from the conceptual model are converted into**



Slide 148

148

CONVERTING LDM INTO PDM

- **Target DBMS known = PostgreSQL**
- **Tables and Columns**
 - Each **table** in the LDM becomes a **CREATE TABLE** statement in PostgreSQL.
 - Each **column** is assigned a concrete PostgreSQL data type.
- **Keys**
 - **Primary Keys (PK):** Implement with PRIMARY KEY constraint.
 - **Surrogate Keys:** Often implemented as SERIAL (auto-increment).
 - **Foreign Keys (FK):** Implement with REFERENCES clause.
 - **Unique Keys:** Add UNIQUE constraint where needed.



Slide 149

149

CONVERTING LDM INTO PDM

- **Relationships**
 - **1:1** : FK with UNIQUE constraint (or merge into one table if always mandatory).
 - **1:N** : FK in the "many" table referencing the "one".
 - **M:N Relationship:** Create a join table with two FKs, both as part of composite PK.
- **Nullability and Defaults**
 - **Optional attributes** => NULL allowed.
 - **Mandatory attributes** => NOT NULL.



Slide 150

150

CONVERTING LDM INTO PDM

- **Constraints and Checks**
 - Enforce **business rules** via:
 - CHECK constraints (eg., orderquantity > 0).
 - UNIQUE constraints for alternate keys.
 - Referential integrity via ON DELETE CASCADE, ON UPDATE RESTRICT.
- **Indexing**
 - **Primary keys** and **unique constraints** automatically create indexes.
 - Add indexes on FKs or frequently queried columns
- **Normalisation vs. Performance**

Slide 151

151

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module A: Tools & Foundations Data Normalisation Recap

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

Slide 152

152

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 153

153

PURPOSE

- Understand the role of normalisation in relational database design.
- Learn how to reduce data redundancy and improve data integrity.
- Bridge theory of normal forms with practical database implementation.

Slide 154

154

OBJECTIVES

- Explain the need for data normalisation in relational databases.
- Apply 1NF, 2NF, 3NF rules to restructure data models.
- Balance normalisation with performance considerations.

Slide 155

155

NORMALISATION

- Set of rules for allocating data to tables in such a way that it eliminates:
 - certain types of redundancy
 - Incompleteness
- Used in Logical Data Modelling Stage
- Two step process:
 - Put the data into tabular form (+ identify PK)
 - Remove duplicated data to separate tables (+ identify PK)
- Sound Conceptual Modeling reduces largely the normalisation effort to check the correctness of the model
- A fully normalised model is free of redundancy
- Denormalisation is adding redundancy for specific reasons during Physical Data Modeling Stage or for specific purpose such as Business Intelligence

Slide 156

156

EXAMPLE: EMPLOYEE FORM

Employee Code	KDV	Employee Name	Katrien Devos	Date of Birth	15/03/1980
Department Code	IT	Department Name	Information Technology	Start Date	1/07/2003
Qualification Name	Year				
Bachelor of Economics	2000				
Master of Economics	2003				
Master of Business Analysis	2007				
Prince2 Foundation	2010				

Slide 157

157

COVERT FORM INTO EMPLOYEE TABLE

Employee Code	Employee Name	Date of Birth	Department Code	Department Name	Start Date	Qualification 1 Name	Qualification 1 Year	Qualification 2 Name	Qualification 2 Year	Qualification 3 Name	Qualification 3 Year	Qualification 4 Name	Qualification 4 Year
KDV	Katrien Devos	15/03/80	IT	Information Technology	1/07/03	Bachelor of Economics	2000	Master of Economics	2003	Master of Business Analysis	2007	Prince 2 Foundation	2010
JP	Jan Peeters	10/12/78	IT	Information Technology	25/09/00	Bachelor of Economics	1998	Master of Informatics	2001				
KDG	Karel De Grootte	21/08/75	LE	Legal	1/10/10	Bachelor of Law	1995	Master of Law	1997				
LJ	Leen Janssen	21/10/81	AC	Accounting	1/03/12	Professional Bachelor of Accountancy	2001						

Relational Notation:

Table Name

EMPLOYEE (Employee Code, Employee Name, Date of Birth, Department Code, Department Name, Start Date, (Qualification Name, Qualification Year))

Primary Key

Repeating Group

Slide 158

158

REMOVE REPEATING GROUPS

Employee Table

Employee Code	Employee Name	Date of Birth	Department Code	Department Name	Start Date
KDV	Katrien Devos	15/03/80	IT	Information Technology	1/07/03
JP	Jan Peeters	10/12/78	IT	Information Technology	25/09/00
KDG	Karel De Grootte	21/08/75	LE	Legal	1/10/10
LJ	Leen Janssen	21/10/81	AC	Accounting	1/03/12

EMPLOYEE (Employee Code, Employee Name, Date of Birth, Department Code, Department Name, Start Date)

Qualification Table

Employee Code	Qualification Name	Qualification Year
KDV	Bachelor of Economics	2000
KDV	Master of Economics	2003
KDV	Master of Business Analysis	2007
KDV	Prince 2 Foundation	2010
JP	Bachelor of Economics	1998
JP	Master of Economics	2001
KDG	Bachelor of Law	1995
KDG	Master of Law	1997
LJ	Professional Bachelor of Accountancy	2001

QUALIFICATION (Employee Code, Qualification Name, Qualification Year)

Slide 159

159

REMOVE DUPLICATED DATA

Employee Table

Employee Code	Employee Name	Date of Birth	Department Code	Start Date
KDV	Katrien Devos	15/03/80	IT	1/07/03
JP	Jan Peeters	10/12/78	IT	25/09/00
KDG	Karel De Grootte	21/08/75	LE	1/10/10
LJ	Leen Janssen	21/10/81	AC	1/03/12

EMPLOYEE (Employee Code, Employee Name, Date of Birth, Department Code, Start Date)

Department Table

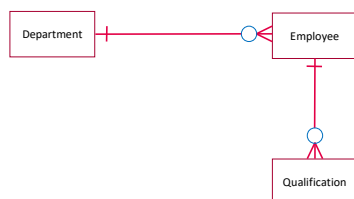
Department Code	Department Name
IT	Information Technology
LE	Legal
AC	Accounting

DEPARTMENT (Department Code, Department Name)

Slide 160

160

POSSIBLE CDM FOR EMPLOYEE FORM



Slide 161

161

RELATIONAL NOTATION WITH ATTRIBUTE DEFINITIONS

- EMPLOYEE
 - Employee Code: 3 Characters – the identifier allocated to this employee by the HR department
 - Employee Name: 60 Characters – the surname, a comma and space, the first given name
 - Date of Birth: the date upon which the employee is born in “dd/mm/yyyy” format
 - Department Number: 3 Characters - the code used by HR to identify the Department that pays this employee’s salary
 - Start Date: the date upon which the employee has started in “dd/mm/yyyy” format

Slide 162

162

CASE STUDY BUILD A RELATIONAL MODEL FOR A MEDICAL SURVEY

Hospital Number	HO1	Hospital Name	UZ Gasthuisberg	Operation Number	33
Hospital Category	AC	Contact at Hospital	Jan Bergmans		
Operation Name	Heart Transplant	Operation Code	38	Procedure Group	Transplant
Surgeon Number		178 Surgeon Speciality	Cardiology	Total Drug Cost	€ 107,50
Drug Code		Name of Drug	Manufacturer	Method	Cost of Dose
ANT 150 mg		Antibiotica	GSK	ORAL	€ 3,00
PEN 250 mg		Penicilin	JNI	IV	€ 3,50
PEN 500 mg		Penicilin	JNI	ORAL	€ 5,00

- One form for each operation
- Unique Hospital Number
- Operation Number is sequence number within one hospital
- Hospital Category can be Private, Public or Academic. Academic Hospitals are Public.
- Operation code is international code
- Surgeon Number is assigned by each Hospital
- One Surgeon recorded per Operation

163

Slide 163

INITIAL MODEL

- OPERATION (Hospital Number, Operation Number, Hospital Name, Hospital Category, Contact Person, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality, Total Drug Cost, Drug Code 1, Drug Name 1, Manufacturer 1, Method 1, Cost of Dose 1, Number of Doses 1, Drug Code 2, Drug Name 2, Manufacturer 2, Method 2, Cost of Dose 2, Number of Doses 2, Drug Code 3, Drug Name 3, Manufacturer 3, Method 3, Cost of Dose 3, Number of Doses 3, Drug Code 4, Drug Name 4, Manufacturer 4, Method 4, Cost of Dose 4, Number of Doses 4)

164

Slide 164

DETERMINE COLUMNS ONE FACT PER COLUMN

- Multifact Columns?
 - o Drug Code: ANT 150 mg
 - Code
 - Dosage
 - Value
 - UoM
 - o Hospital Category: AC
 - Hospital Type: PU
 - Research & Education: YES

165

Slide 165

DETERMINE COLUMNS HIDDEN DATA

- Hidden Data?
 - o Type of Form used (colour coded for specific Operations, etc)
 - o Order of form return
 - o Date of form return

166

Slide 166

DETERMINE COLUMNS DERIVED DATA

- No Redundant data
- Derived data?
 - o Total Drug Cost = Cost of Dose * Number of Doses

167

Slide 167

DETERMINE COLUMNS DETERMINE PRIMARY KEY

- Each value of the PK uniquely defines one row within the table.
 - o Can be defined as the *minimal* set of columns that contain a different combination of values for each row of the table
 - o Can be defined by a unique sequential number
- What is a possible PK?
 - o Hospital Number + Operation Number
 - o Could there ever be more than one form with the same combination of Hospital Number and Operation Number?
 - o What if we repeat this survey every year and want to compare results for the same hospital and operation across different years?

168

Slide 168

163

164

165

166

167

168

DETERMINE COLUMNS

- OPERATION (Hospital Number, Operation Number, Hospital Name, Hospital Type, Research & Education, Contact Person, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality, Drug Short Name 1, Drug Name 1, Manufacturer 1, Size of Dose 1, Unit of Measure 1, Method 1, Cost of Dose 1, Number of Doses 1, Drug Short Name 2, Drug Name 2, Manufacturer 2, Size of Dose 2, Unit of Measure 2, Method 2, Cost of Dose 2, Number of Doses 2, Drug Short Name 3, Drug Name 3, Manufacturer 3, Size of Dose 3, Unit of Measure 3, Method 3, Cost of Dose 3, Number of Doses 3, Drug Short Name 4, Drug Name 4, Manufacturer 4, Size of Dose 4, Unit of Measure 4, Method 4, Cost of Dose 4, Number of Doses 4)

169

1NF - REPEATING GROUPS

- Repeating Groups:
 - Inflexible
 - Complex to retrieve and maintain
- STEP 1 – Put the data in table form by identifying and eliminating Repeating Groups
- Drug Short Name 1, Drug Name 1, Manufacturer 1, Size of Dose 1, Unit of Measure 1, Method 1, Cost of Dose 1, Number of Doses 1, Drug Short Name 2, Drug Name 2, Manufacturer 2, Size of Dose 2, Unit of Measure 2, Method 2, Cost of Dose 2, Number of Doses 2, Drug Short Name 3, Drug Name 3, Manufacturer 3, Size of Dose 3, Unit of Measure 3, Method 3, Cost of Dose 3, Number of Doses 3, Drug Short Name 4, Drug Name 4, Manufacturer 4, Size of Dose 4, Unit of Measure 4, Method 4, Cost of Dose 4, Number of Doses 4
- {Drug Short Name, Drug Name, Manufacturer, Size of Dose, Unit of Measure, Method, Cost of Dose, Number of Doses}

170

1NF - REPEATING GROUPS

- Remove each set of Repeating Group columns to a new table. One Group occurrence becomes a row in new table;
- Give new table a meaningful name;
- Include the key of the original table in the new table (foreign key);
- If applicable add sequence column;
- Identify the Primary Key of the new table.

171

1NF - REPEATING GROUPS

- OPERATION (Hospital Number, Operation Number, Hospital Name, Hospital Type, Research & Education, Contact Person, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality)
- DRUG PRESCRIPTION (Hospital Number, Operation Number, Drug Short Name, Size of Dose, Unit of Measure, Method, Drug Name, Manufacturer, Cost of Dose, Number of Doses)
- What about the sequence of prescription in the survey form?
- If we add what would be the new PK?

172

1NF AND CDM

173

2NF – REDUNDANT FACTS

- OPERATION (Hospital Number, Operation Number, Hospital Name, Hospital Type, Research & Education, Contact Person, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality)
- DRUG PRESCRIPTION (Hospital Number, Operation Number, Drug Short Name, Size of Dose, Unit of Measure, Method, Drug Name, Manufacturer, Cost of Dose, Number of Doses)
- Issues
 - Changing any of these facts will require an update of all records in the table
 - Deleting a row with only one occurrence will remove all information

174

2NF – REDUNDANT FACTS

- Hospital Name, Hospital Type, Research & Education, Contact Person are functionally dependent of Hospital Number.

○ Hospital Number is a determinant of the other columns.

○ Hospital Number -> Hospital Name, Hospital Type, Research & Education, Contact Person

Slide 175

175

2NF – REDUNDANT FACTS

- Identify any determinant part of the tables PK and the column(s) they determine;
- Define a new table for each determinant and the columns it determines. The determinant becomes the PK of that table;
- Give new table a meaningful name;
- Remove the determined columns from original table.
- A table is in 2NF if
 - it is already in 1NF, and
 - every attribute in the table is functionally determined by the whole identifier (and not just a part of the identifier)

Slide 176

176

2NF – REDUNDANT FACTS

- OPERATION (Hospital Number, Operation Number, Hospital Name, Hospital Type, Research & Education, Contact Person, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality)
- HOSPITAL (Hospital Number, Hospital Name, Hospital Type, Research & Education, Contact Person)

Slide 177

177

2NF – REDUNDANT FACTS

- DRUG PRESCRIPTION (Hospital Number, Operation Number, Drug Short Name, Size of Dose, Unit of Measure, Method, Drug Name, Manufacturer, Cost of Dose, Number of Doses)
- Drug Short Name -> Drug Name, Manufacturer
- Drug Short Name, Size of Dose, Unit of Measure, Method -> Cost of Dose

Slide 178

178

2NF – REDUNDANT FACTS

- DRUG PRESCRIPTION (Hospital Number, Operation Number, Drug Short Name, Size of Dose, Unit of Measure, Method, Drug Name, Manufacturer, Cost of Dose, Number of Doses)
- DRUG (Drug Short Name, Drug Name, Manufacturer)
- STANDARD DRUG DOSE (Drug Short Name, Size of Dose, Unit of Measure, Method, Cost of Dose)

Slide 179

179

3NF – REDUNDANT FACTS

- OPERATION (Hospital Number, Operation Number, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality)
- Transitive functional dependency (A->B & B->C then also A->C)
 - Operation Code -> Operation Name, Procedure Group
 - Hospital Number, Operation Number -> Operating Code
 - Hospital Number, Operation Number -> Operation Name, Procedure Group
 - Hospital Number, Surgeon Number -> Surgeon Speciality
 - Hospital Number, Operation Number -> Hospital Number, Surgeon Number
 - Hospital Number, Operation Number -> Surgeon Speciality

Slide 180

180

3NF – REDUNDANT FACTS

- OPERATION (Hospital Number, Operation Number, Operation Name, Operation Code, Procedure Group, Surgeon Number, Surgeon Speciality)
- OPERATION TYPE (Operation Code, Operation Name, Procedure Group)
- SURGEON (Hospital Number, Surgeon Number, Surgeon Speciality)

181

Slide 181

181

3NF – REDUNDANT FACTS

- Identify any determinant not part of the tables PK and the column(s) they determine;
- Define a new table for each determinant and the columns it determines. The determinant becomes the PK of that table;
- Give new table a meaningful name;
- Remove the determined columns from original table.
- A table is in 3NF if
 - it is already in 2NF, and
 - there are no transitive dependencies.

182

Slide 182

182

3NF – FINAL RELATIONAL MODEL

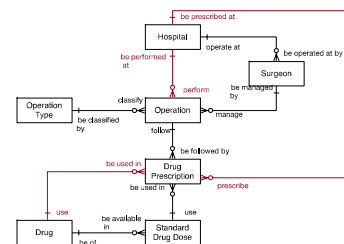
- OPERATION (Hospital Number, Operation Number, Operation Code, Surgeon Number)
- OPERATION TYPE (Operation Code, Operation Name, Procedure Group)
- SURGEON (Hospital Number, Surgeon Number, Surgeon Speciality)
- HOSPITAL (Hospital Number, Hospital Name, Hospital Type, Research & Education, Contact Person)
- DRUG PRESCRIPTION (Hospital Number, Operation Number, Drug Short Name, Size of Dose, Unit of Measure, Method, Number of Doses)
- DRUG (Drug Short Name, Drug Name, Manufacturer)
- STANDARD DRUG DOSE (Drug Short Name, Size of Dose, Unit of Measure, Method, Cost of Dose)

183

Slide 183

183

3NF AND CDM



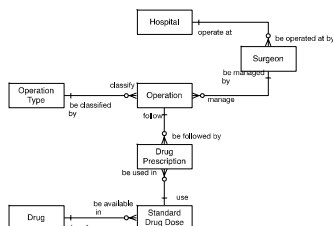
184

Slide 184

184

3NF AND CDM

WITHOUT REDUNDANT RELATIONS



185

Slide 185

185

FURTHER NORMALISATION

- A relational model in 3NF may not be fully normalised.
- Further normalisation might be necessary
 - Boyd-Codd Normal Form
 - 4NF
 - 5NF
- In practice normalisation is used as check on the correctness of a model developed using a top down approach.

186

Slide 186

186